LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ***??***

SUMMARY We analyze an authorization decision inconsistency in SAGA. A formal coverage diagnostic first shows that existing authentication and token-secrecy properties do not establish a required authorization correspondence. Implementation analysis then identifies an order-dependent matcher defect whose decision propagates through production consumers to token issuance and storage. Under the fixed divergence, ProVerif reports `ChatAccept` reachable; separately, ProVerif verifies a non-injective correspondence requiring prior `TokenIssue`.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

LLM-based autonomous agents increasingly operate with external tools and services [1], [2]. Authentication establishes the identity of a communicating agent, but it does not determine whether that agent is permitted by the receiver's contact policy. Consequently, secure agent communication requires not only authenticated participants and protected credentials, but also authorization decisions consistent with the receiver's intended policy. SAGA [3] combines authenticated identities, contact policies, and access-control tokens. In line with the distinction between authentication and authorization [4], its verified authentication and token-secrecy properties do not establish that token issuance follows the receiver's intended policy.

As a first stage, an authorization coverage diagnostic of SAGA's reference communication model retains the existing authentication and token-secrecy results but reports an added authorization correspondence as false. This exposes a proof-coverage gap: the diagnostic neither executes the Python matcher nor identifies its defect. It instead motivates our inspection of policy evaluation and enforcement, which identifies an order-dependent inconsistency in the production matcher.

Controlled matcher executions establish the resulting Spec-Deny/Impl-Allow decision, and consumer validation traces it through production Provider and receiver paths to token issuance and storage. The Decision Bridge transfers this verified decision pair to a fixed ProVerif scenario; it is neither a generic code-to-model translator nor a refinement proof. In that scenario, `ChatAccept` is reachable while its prior-`TokenIssue` correspondence remains true. Together, these evidence layers characterize an authorization-bypass path. The empirical path stops at token issuance and storage; only the symbolic model reaches message acceptance.

[†]TODO: affiliation and postal address.

DOI: 10.1587/transfun.E0.A.1

Our contributions are (1) diagnosing an authorization proof-coverage gap in SAGA's reference model; (2) identifying the order-dependent production matcher inconsistency and validating its propagation through production consumers to token issuance and storage; and (3) showing that, under the same fixed Spec-Deny/Impl-Allow divergence, symbolic `ChatAccept` is reachable while the prior-`TokenIssue` correspondence remains true.

(1) Related work.

Prior work studies distributed authorization languages and policy semantics [5], [6] and policy verification and change-impact analysis [7]. SPKI [4] and Macaroons [8] provide related abstractions for credential- and capability-based authorization. Symbolic protocol analyzers ProVerif and Tamarin support protocol-level security verification [9], [10]; SAGA uses ProVerif to analyze its communication protocol [3]. Security evaluation for autonomous LLM agents has also received recent attention [2]. We instead examine a concrete deviation between SAGA's specified rule-selection semantics and its implementation and trace its downstream consumer consequences, complementing SAGA's protocol-level guarantees under its model assumptions.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

An initiator requests contact through SAGA's Provider. The receiver's ordered policy maps identity patterns to integer budgets. The Provider and receiver independently evaluate the initiator using the production matcher. The Provider gates one-time key (OTK) release; the receiver rechecks its policy before token issuance.

For a fixed policy and valid identity, SAGA, Section IV-D.1 [3], specifies that the most-specific matching rule is selected; the implementation scores patterns with `aid_specificity()`. A positive budget permits the request; a negative budget denies it. We do not consider equal-specificity conflicts, policy changes during evaluation, or zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`. The accompanying artifact records the pinned upstream source provenance and binds the

Copyright © 200x The Institute of Electronics, Information and Communication Engineers

Table 1 Specification and production-matcher results under two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	−1	10
Wildcard allow, then specific deny	−1	−1

archived matcher observation to that source.† Its `best_pattern` starts at `None`, whose specificity is −1. A matching rule with greater specificity updates the budget but never `best_pattern`. Thus later matching rules still compare against −1, allowing even a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and * as 0; their budgets are −1 and 10, respectively.

A separate finite specification evaluator selects the specific denial for both orders in Table 1. When the specific deny rule precedes the wildcard-allow rule, the specification denies with budget −1, but the production matcher returns 10 and permits. The reverse order returns the specified denial, establishing order dependence. This constitutes an implementation-level authorization bypass in the tested case: the specification denies while the production matcher permits. These matcher calls alone do not establish downstream token state, message acceptance, or a deployed exploit.

3. Formal Authorization Analysis

(1) Coverage diagnostic.

Our analysis begins with an authorization coverage diagnostic of SAGA's reference communication model. The archived ProVerif 2.05 extension preserves the original authentication and token-secrecy results, whereas an added correspondence requiring `Accept` to have prior `PolicyAllow` is false. Here `Accept` means token receipt and verification, not the later `ChatAccept`. The diagnostic does not connect its policy events to an enforcement guard; it exposes a missing authorization proof obligation and neither executes the matcher nor identifies the Python defect.

(2) Consequence model.

The later symbolic model fixes the recorded divergence: the specification denies and the observed implementation permits. ProVerif assumes these facts; it does not run Python or the production matcher, discover the inconsistency, or establish implementation refinement.

(3) Model assumptions.

The initiator has a valid registered identity, follows the prescribed symbolic peer process, and cannot alter the receiver's fixed policy. Reachability under these assumptions is therefore not an arbitrary-adversary reachability claim. The Provider and Certificate Authority (CA) are trusted, cryptographic checks are effective, and private keys remain uncompromised; a private symbolic channel abstracts authenticated, confidential transport.

In the symbolic receiver, $\texttt{TokenIssue}(R, I, t)$ occurs after an `ActiveToken` entry stores receiver R, initiator I, encrypted token t, and I's public access-control key, but before t is sent. For private-channel message m, $\texttt{ChatAccept}(R, I, t, m)$ occurs only after checking the fixed identity and an `ActiveToken` entry matching receiver R, initiator I, presented token t, and the sender public key associated with I.

(4) Authorization conditions and results.

The accompanying artifact archives the ProVerif 2.05 run, generated model, and raw output. The run reports `ChatAccept` reachable under the fixed divergence. Separately, it reports this non-injective correspondence as true:

$$\mathsf{ChatAccept}(R, I, t, m) \Longrightarrow \mathsf{TokenIssue}(R, I, t)$$

Here R is the receiver and token issuer, and I is the initiator. The correspondence requires a prior `TokenIssue` for the same R, I, and encrypted token t used by `ChatAccept`; it establishes token-origin consistency. A distinct ideal authorization condition for a static receiver policy would be

$$\mathsf{TokenIssue}(R, I, t) \Longrightarrow \mathsf{Allow}_{\mathrm{spec}}(P_R, I),$$

where P_R is R's contact policy and $\mathsf{Allow}_{\mathrm{spec}}$ denotes its intended semantics, rather than the matcher's implementation result. This second formula is neither an executed ProVerif query nor a proved theorem; it states the desired authorization condition. Token-origin consistency therefore does not imply policy-compliant issuance.

For the recorded specific-Deny plus wildcard-Allow rules, the intended policy denies while the matcher permits. Under this fixed divergence, `ChatAccept` is reachable while the token-origin correspondence remains true, so correct token origin coexists with intended-policy denial. The `ChatAccept` witness includes token issuance; no standalone `TokenIssue` reachability query exists.

The empirical and symbolic executions are compared only at the level of authorization decisions and token-lifecycle events; no object-level refinement is established. Only the symbolic execution reaches message acceptance.

(5) Authorization-bypass path.

Across the evidence layers, these results characterize an authorization-bypass path: the documented policy denies the initiator, the production matcher permits it, the decision propagates through Provider and receiver processing to token issuance and storage, and the corresponding fixed symbolic divergence admits `ChatAccept`. The implementation and symbolic endpoints remain separate evidence layers. We use "authorization bypass" for this policy-enforcement failure, not to claim an arbitrary-adversary or deployed exploit.

†Reproduction artifact: `github.com/g1020268262-a11y/SAGA-Authorization-Analysis-Artifact`.

Table 2 Divergent-case propagation through production consumer paths; message acceptance was not executed.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider `/access`	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

4. Consumer-Side Propagation Validation

(1) Purpose.

This experiment validates the implementation segment of the authorization-bypass path. It follows the recorded Spec-Deny/Impl-Allow case through the production Provider and receiver consumers to token issuance and storage; it does not execute application-message acceptance.

(2) Environment.

The archived consumer run records source hashes for the matcher, Provider, receiver, and token-related cryptographic code. These relevant sources match the pinned upstream baseline used in the analysis. The harness drives the production Provider `/access` route and receiver handler from matcher evaluation through `active_tokens` storage. The harness supplies signed registration material, local certificates, and in-memory transport while retaining the production cryptographic checks. Matcher wrappers invoke the original matcher, record its result, and return that result unchanged; fixtures preset neither authorization decisions nor the resulting token state. A boundary stub intercepts the call before the original `receive_conversation` executes.

(3) Cases and controls.

The Provider and receiver use identical policies. The divergent D/A case places the specific deny rule (−1) before the wildcard-allow rule (10). The controls are A/A (both rules allow with 10) and D/D (the wildcard-allow rule 10 before the specific deny rule −1). Each case ran once with fresh state and one OTK. The first and second letters denote the specification and observed implementation decisions, respectively, rather than Provider and receiver outcomes. A/A reaches token sending and storage; D/D returns HTTP 403 with no OTK release or state change and never reaches the receiver, so receiver rejection remains untested.

(4) Result and evidence.

Table 2 shows that both matchers return 10, the OTK is released and consumed, and one token is generated, sent, and stored in `active_tokens`. The endpoint excludes message acceptance: no application message is sent, and the original `receive_conversation` is not invoked.

5. Scope and Conclusion

Our analysis proceeds from a formal authorization-coverage diagnostic to a concrete implementation defect and then to its authorization-bypass consequences. The diagnostic shows that the existing authentication and token-secrecy results do not establish the required policy-authorization obligation. Implementation inspection then identifies a production matcher violation of the documented most-specific semantics. In controlled executions, the resulting decision reaches token issuance and storage; under the corresponding fixed symbolic divergence, `ChatAccept` is reachable even though its prior-`TokenIssue` correspondence remains true. Token-origin consistency can therefore coexist with specification denial.

The evidence covers one recorded SAGA case and the tested policies. It does not establish arbitrary-policy correctness, Python–ProVerif refinement, an arbitrary-attacker exploit, or a deployed exploit. Empirical execution stops before application-message acceptance; that endpoint belongs only to the symbolic model.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W.X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol.18, no.6, p.186345, 2024.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," Advances in Neural Information Processing Systems 37 (NeurIPS 2024), pp.82895–82920, 2024.

[3] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," Network and Distributed System Security (NDSS) Symposium, Internet Society, Feb. 2026.

[4] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, "SPKI certificate theory," RFC 2693, RFC Editor, Sept. 1999. Experimental.

[5] M.Y. Becker and P. Sewell, "Cassandra: Distributed access control policies with tunable expressiveness," 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY), 2004.

[6] M.Y. Becker, C. Fournet, and A.D. Gordon, "Design and semantics of a decentralized authorization language," 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.

[7] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, "Verification and change-impact analysis of access-control policies," 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.

[8] A. Birgisson, J.G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," Network and Distributed System Security Symposium (NDSS), Internet Society, 2014.

[9] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.

[10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," Computer Aided Verification (CAV), Lecture Notes in Computer Science, vol.8044, pp.696–701, Springer, 2013.